

Human Vision – Topic 1

Computational Vision

Yonghao Lee

April 20, 2026

Contents

1	Introduction	2
1.1	Computational Vision vs. Computer Vision	2
1.2	Marr’s Three Levels of Analysis	2
1.3	Worked Example: Catching a Mosquito	3
2	Psychophysics of Motion	3
2.1	Infinite Lines and the Aperture Problem	3
2.2	The Constraint Line	4
2.3	Phenomenon 1: Humans Perceive the Normal Velocity	5
2.4	Phenomenon 2: The Barberpole Effect	5
2.5	Terminator Velocity	6
2.6	Plaids: Combining Two Constraint Lines	6
3	Motion Estimation as a Computational Problem	7
3.1	Problem Formulation	7
3.2	Generalization to Realistic Scenes	8
3.3	The Constant Brightness Assumption	8
3.4	Sum of Squared Differences (SSD)	8
4	SSD and the Constraint Line: Two Views of One Object	9
4.1	A Single Edge: SSD Is Flat Along the Constraint Line	9
4.2	Plaids and Corners: SSD Has a Unique Minimum at the IOC Velocity . . .	9
5	The Lucas–Kanade Algorithm	10
5.1	Linearizing SSD via Taylor Expansion	10
5.2	The Normal Equations	11
5.3	The Aperture Problem in Matrix Form	11
5.4	Limitations and Practical Extensions	12
5.5	Lucas–Kanade in Marr’s Framework	12
6	Summary	13

1 Introduction

1.1 Computational Vision vs. Computer Vision

Computer vision is an engineering discipline that asks, “How can we build machines that see?” **Computational vision** is a scientific one that asks, “How can we model and understand how humans see?” The two fields share a great deal of mathematical machinery but differ in what counts as a successful answer: an engineer wants a working system, a scientist wants an explanation of human perception.

Traditionally, questions about human vision have been addressed through neuroscience, where explanations focus on the physical implementation in the brain. Three common paradigms are:

- **Single-cell paradigm.** Insert microelectrodes into the visual cortex and identify individual neurons that respond to specific stimuli (e.g. a cell that fires for a 30° edge but not for a 60° edge).
- **Cell circuits.** The same idea, extended to networks of neurons and their interactions.
- **Brain areas.** Neuroimaging (such as fMRI) maps the cortex into functionally specialized regions—e.g. areas responding selectively to faces or to spatial layout.

This course will examine three aspects of the human visual system: motion processing, color processing, and form processing.

1.2 Marr’s Three Levels of Analysis

David Marr argued that trying to understand perception by studying only neurons is like trying to understand bird flight by studying only feathers: it cannot be done. To understand bird flight we need aerodynamics; only then do the structure of feathers and the shapes of birds’ wings make sense. Marr proposed that any information-processing system must be analyzed at three distinct levels.

Definition 1.1 Marr’s Three Levels

1. **Computational level.** What is the goal of the computation, why is it appropriate, and what is the logic of the strategy by which it can be carried out? In short: *what* is the system trying to achieve, and *why*?
2. **Representation and algorithm level.** How can the computational theory be implemented? What is the representation of the input, and what algorithm operates on it?
3. **Implementation level.** How is the representation and algorithm realized physically?

Course Focus

Computational vision focuses heavily on Marr’s **computational level**: what problem is the visual system solving, why is it hard, and what assumptions about the world make it solvable?

1.3 Worked Example: Catching a Mosquito

Luke and Han are trying to catch a mosquito. Each time they raise a hand, the mosquito flies away. They disagree on why.

- **Luke:** “The mosquito has a neuron in its visual cortex that fires whenever a hand enters its visual field.” This explanation is stated in terms of biological hardware and cellular response—the **implementation level**.
- **Han:** “The mosquito’s visual system computes a dot product between the input image and a template of a hand; if the dot product exceeds a threshold, the mosquito flies away.” This is a mathematical procedure with an explicit representation, but no commitment to how the brain physically carries it out—the **representation and algorithm level**.

2 Psychophysics of Motion

Psychophysics studies the mapping from physical stimuli (ϕ) to psychological perception (Φ). For example, one might turn up the physical volume of a speaker and measure how much louder a subject actually perceives it to be.

To study motion perception we simplify the stimulus: we focus on two-dimensional motion described by a velocity vector (v_x, v_y) .

2.1 Infinite Lines and the Aperture Problem

Wallach (1935) observed that the motion of an “infinite” line (or a line viewed through an aperture that hides its endpoints) is fundamentally ambiguous. When the endpoints are visible, the line’s direction of motion is easy to determine. Without them, many distinct physical motions produce the exact same visual stimulus.

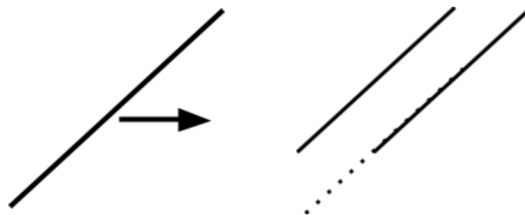


Figure 1: Ambiguity of infinite line movement.

The key insight is a decomposition of the velocity vector.

Definition 2.1 Decomposition of the Velocity Vector

Any 2D velocity vector $\vec{v}$ of a line can be decomposed into two orthogonal components:

$$\vec{v} = \vec{v}^\perp + \vec{v}^\parallel.$$

1. **Normal velocity** ($\vec{v}^\perp$): the component perpendicular to the line.
2. **Parallel velocity** ($\vec{v}^\parallel$): the component parallel to the line.

The parallel component is invisible: shifting an infinite line along its own axis leaves it looking identical. Infinitely many velocity vectors therefore produce the same visual motion.

The Aperture Problem, Stated Precisely

Let M be the movie generated by an infinite line of orientation θ moving with velocity $\vec{v}$. Any alternative velocity $\vec{v}'$ with $\vec{v}'^\perp = \vec{v}^\perp$ produces exactly the same movie M . The visual system sees M and must recover $\vec{v}$, but M alone does not determine it—a fundamental ambiguity.

2.2 The Constraint Line

Definition 2.2 Constraint Line

For a line of orientation θ moving with velocity $\vec{v}$, the **constraint line** is the set of all velocities in velocity space (v_x, v_y) that share the same normal component $\vec{v}^\perp$.

Letting $\hat{n}$ denote the unit normal to the line, the constraint line is defined by

$$\vec{v} \cdot \hat{n} = c, \quad c = \|\vec{v}^\perp\|.$$

Geometrically, this is a straight line in velocity space, perpendicular to $\hat{n}$ and therefore *parallel to the edge itself*.

Examples

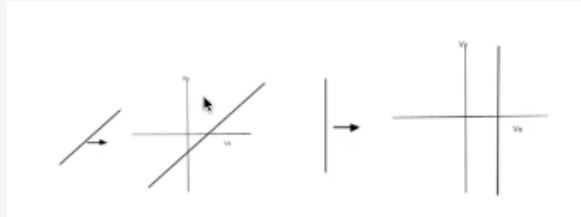


Figure 2: Constraint line in velocity space.

- **Vertical edge moving right.** The normal is horizontal, so the constraint is $v_x = c$: a *vertical* line in velocity space.
- **Edge at 45° (along $(1, 1)$).** The unit normal is $\hat{n} = (1, -1)/\sqrt{2}$, so the constraint is $v_x - v_y = c\sqrt{2}$: a *diagonal* line of slope 1, parallel to the edge.

The Constraint Line as the Aperture Problem

The constraint line is the geometric expression of the aperture problem: a single local measurement gives one linear equation in two unknowns (v_x, v_y) , so the true velocity is underdetermined—it could be any point on the constraint line. Among these, the normal velocity $\vec{v}^\perp$ is the unique point closest to the origin.

2.3 Phenomenon 1: Humans Perceive the Normal Velocity

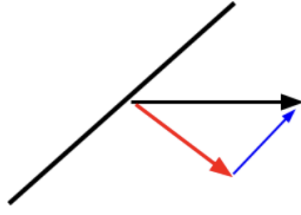


Figure 3: Phenomenon 1. Black arrow: true velocity $\vec{v}$. Red arrow: perceived velocity $\vec{v}^\perp$.

When the stimulus is an infinite line moving with true velocity $\vec{v}$, human observers perceive the motion as $\vec{v}^\perp$.

Why the Normal Velocity?

Among all velocities consistent with the observed movie, $\vec{v}^\perp$ has the smallest magnitude. The visual system appears to apply a **slowest-velocity prior**: when many velocities could explain the stimulus, it selects the slowest.

Phenomenon 1 in Marr's Framework

- **Problem:** estimate the direction of motion.
- **Difficulty:** the aperture problem—infinately many velocities are consistent with the stimulus.
- **Constraint used to resolve it:** select the slowest consistent velocity.

2.4 Phenomenon 2: The Barberpole Effect

A classic demonstration: <https://www.liverpool.ac.uk/~marcob/Trieste/barberpole.html>.

The same drifting grating viewed through apertures of different shapes produces different perceived motion. Through a rectangular aperture, the pattern appears to move along the long axis of the rectangle; through a circular aperture, it appears to move diagonally (in the direction of the normal velocity).

Definition 2.3 Barberpole Effect

For a line of orientation θ moving through a rectangular aperture whose long axis is at orientation θ_A , the perceived direction of motion is θ_A .

2.5 Terminator Velocity

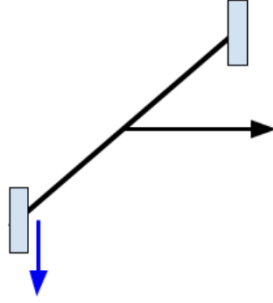


Figure 4: Terminator velocity. Black arrow: true velocity of the line. Blue arrow: velocity of the intersection (terminator) along the aperture boundary.

Definition 2.4 Terminator Velocity

For a line of orientation θ that intersects a static edge of orientation θ_2 , the **terminator velocity** $\vec{v}^T$ is the velocity of the intersection point. It is always directed along θ_2 .

In Figure 4, the line moves horizontally (black arrow), yet the intersection travels along the orientation of the static edge (blue arrow). In the barberpole setup, the two long sides of the rectangle produce terminators moving along the rectangle’s long axis; only the two short sides produce terminators moving differently.

Phenomenon 3: Wallach’s Claim

For a line of orientation θ moving through an aperture with velocity $\vec{v}$, let $\vec{v}^{Ti}$ be the velocity of the i -th terminator. If there is a *dominant* terminator velocity—that is, if $\vec{v}^{Ti} = \vec{v}^*$ for most i —then the perceived velocity is $\vec{v}^*$.

Remark

This principle explains the rectangular-aperture case: the dominant terminator velocity is along the long axis, so that is what we perceive. It does not explain the circular-aperture case, where no single terminator direction dominates.

2.6 Plaids: Combining Two Constraint Lines

A **plaid** is formed by superimposing two gratings of different orientations that move together as a single rigid pattern. Unlike a single line, a plaid provides *two* edges with different orientations, and hence two constraint lines in velocity space. The natural question is: how does the visual system combine the two local measurements into a single perceived velocity?

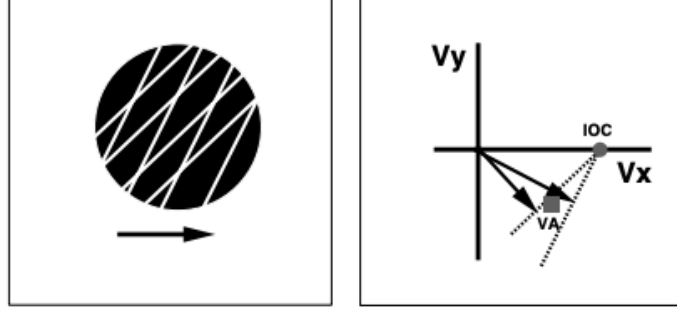


Figure 5: Plaid stimulus (left) and the two candidate combination rules in velocity space (right): vector average (VA) and intersection of constraints (IOC).

Definition 2.5 Vector Average (VA)

The perceived velocity is the average of the two normal velocities:

$$\vec{v}_{VA} = \frac{1}{2}(\vec{v}_1^\perp + \vec{v}_2^\perp).$$

This rule is computationally simple but generally does not recover the true motion.

Definition 2.6 Intersection of Constraints (IOC)

The perceived velocity is the unique point in velocity space where the two constraint lines intersect:

$$\vec{v}_{IOC} = C_1 \cap C_2.$$

This is the only velocity simultaneously consistent with both observations, and it recovers the true 2D velocity of the plaid.

Why Two Rules?

The IOC rule is mathematically correct: two constraint lines uniquely determine the velocity, resolving the aperture problem. The VA rule is simpler but generally wrong. Later phenomena will test which rule the human visual system actually uses, and under what conditions.

3 Motion Estimation as a Computational Problem

We now turn to the algorithmic side: given a sequence of images, how can a computer recover the velocity at which the underlying scene content is moving?

3.1 Problem Formulation

We begin with a deliberately simplified version:

- **Input:** two consecutive frames $I_1(x, y)$ and $I_2(x, y)$, taken a short time apart.
- **Output:** a single velocity vector (v_x, v_y) describing how the content of the image moved between the two frames.

In this setting we assume the entire image moves as a single rigid unit; the two frames are related by an overall translation, and our task is to estimate it.

3.2 Generalization to Realistic Scenes

Real scenes are not globally translating—different objects move independently, and parts of the scene (such as the background) may not move at all. The standard remedy is to apply the simplified two-frame problem *locally*: divide the image into small patches and estimate a separate velocity for each. The patch-wise estimates together form a dense motion field over the image.

3.3 The Constant Brightness Assumption

To relate the two frames we need a modeling assumption linking their intensity values. The standard choice is the **constant brightness assumption**:

$$I_2(x + v_x, y + v_y) = I_1(x, y).$$

In words: a point in the scene at location (x, y) in frame 1 appears at location $(x + v_x, y + v_y)$ in frame 2, with *unchanged brightness*. Only its location has shifted.

Reading the Equation

I_1 and I_2 are two fixed photos. The left side looks up the brightness of frame 2 at the *shifted* location; the right side looks up the brightness of frame 1 at the *original* location. The equation asserts that these two brightness readings must be equal—because they are reports of the same physical point at two different times.

This assumption is only approximately true. It is violated by lighting changes, shadows, specular reflections, occlusions, and image noise. Nevertheless, it is the foundation on which most classical motion estimation methods are built.

3.4 Sum of Squared Differences (SSD)

Since the constant brightness equation rarely holds exactly, we cannot solve it directly. Instead, we seek the velocity that makes it hold *as closely as possible* over a region of the image, by minimizing the **sum of squared differences**:

$$\text{SSD}(v_x, v_y) = \sum_{x,y} (I_2(x + v_x, y + v_y) - I_1(x, y))^2,$$

and taking as our estimate

$$v^* = \arg \min_v \text{SSD}(v_x, v_y).$$

What SSD Actually Does

SSD is a function of a *candidate* velocity, evaluated on two fixed frames. Operationally: pick (v_x, v_y) , shift I_2 by it, and compare the shifted I_2 to I_1 pixel by pixel; the sum of squared differences is the score. Small score means the candidate shift made I_2 match I_1 well; large score means it did not. The velocity that minimizes this mismatch is our estimate of the true motion.

4 SSD and the Constraint Line: Two Views of One Object

The constraint line was introduced psychophysically, as the set of velocities that look identical to a human observer viewing an infinite line. We now show that this same set arises algorithmically as the set of velocities where the SSD cost function is zero. The psychophysical and algorithmic definitions describe the same object.

4.1 A Single Edge: SSD Is Flat Along the Constraint Line

From Constraint Line to Zero SSD

If the image contains an infinite line, every velocity on the constraint line yields $\text{SSD} = 0$. The argument is a three-step chain:

$$\underbrace{\vec{v}' \text{ on the constraint line}}_{\text{same normal component as } \vec{v}} \implies \underbrace{\vec{v}' \text{ produces the same movie as } \vec{v}}_{\text{aperture problem}} \implies \underbrace{\text{SSD}(\vec{v}') = 0}_{\text{pixelwise match}}.$$

Derivation. Let $\vec{v}$ be the true velocity, so that $I_2(x + v_x, y + v_y) = I_1(x, y)$. Pick any $\vec{v}'$ on the constraint line; by definition $\vec{v}'^\perp = \vec{v}^\perp$, and shifting an infinite line by velocities with the same normal component produces the same image. Hence $I_2(x + v'_x, y + v'_y) = I_1(x, y)$ at every pixel, and

$$\text{SSD}(v'_x, v'_y) = \sum_{x,y} (I_2(x + v'_x, y + v'_y) - I_1(x, y))^2 = 0.$$

Two Equivalent Characterizations of the Constraint Line

- **Psychophysical (geometric):** all velocities sharing the same normal component $\vec{v}^\perp$; the velocities that look identical on an infinite line.
- **Algorithmic (SSD):** all velocities along which the SSD cost is zero; the velocities that an SSD-based estimator cannot tell apart.

Both the human visual system and an SSD-based algorithm are blind to motion along the edge.

4.2 Plaids and Corners: SSD Has a Unique Minimum at the IOC Velocity

We now consider the opposite case: an image whose patch contains edges at multiple orientations (a plaid, a corner, or a textured region). Here the SSD landscape has a unique minimum, and it coincides with the IOC velocity.

The Observation

For an image containing edges at several different orientations moving as a rigid pattern, the IOC velocity is the unique minimizer of SSD:

$$\vec{v}_{\text{IOC}} = \arg \min_v \text{SSD}(v_x, v_y), \quad \text{SSD}(\vec{v}_{\text{IOC}}) = 0.$$

Why this holds. SSD is a sum of non-negative per-pixel contributions:

$$\text{SSD}(v_x, v_y) = \sum_{x,y} \underbrace{(I_2(x + v_x, y + v_y) - I_1(x, y))^2}_{\text{contribution from pixel } (x,y)}.$$

For the total to vanish, *every* contribution must vanish simultaneously. Grouping pixels by the edge on which they lie, this requires every edge to match between I_1 and the shifted I_2 at once. For edge i with unit normal $\hat{n}_i$, matching requires the candidate $\vec{v}$ to lie on its constraint line C_i . Therefore

$$\{\vec{v} : \text{SSD}(\vec{v}) = 0\} = \bigcap_i C_i.$$

For a rigid pattern with two or more differently-oriented edges, the constraint lines are not parallel and meet at a single point—the IOC velocity. This is the unique zero of SSD, and hence the minimizer v^* .

The Two Regimes

SSD behaves in two qualitatively different ways depending on the image content:

- **Single edge:** SSD is zero along the entire constraint line. Infinitely many velocities tie for the minimum; motion is unrecoverable from local information.
- **Plaid, corner, or textured patch:** SSD is zero only at the intersection of the constraint lines. A unique minimizer exists and equals the IOC velocity.

This is why local motion estimators in computer vision deliberately select corner-like patches (e.g. Shi–Tomasi’s “good features to track”): they are exactly the patches where SSD has a well-defined minimum.

Convergence of the Two Perspectives

IOC was introduced psychophysically, as the way humans resolve plaid motion. We now see it is also what an SSD-based algorithm computes. The psychophysical rule and the algorithmic optimizer agree: a point of genuine contact between Marr’s computational and algorithmic levels.

5 The Lucas–Kanade Algorithm

Minimizing $\text{SSD}(v_x, v_y)$ as stated is awkward: the unknowns sit inside the image function $I_2(x + v_x, y + v_y)$, making the cost non-linear and costly to optimize by brute force. The **Lucas–Kanade algorithm** sidesteps this by linearizing the problem under the assumption that the velocity is small.

5.1 Linearizing SSD via Taylor Expansion

If (v_x, v_y) is small, the first-order Taylor expansion gives

$$I_2(x + v_x, y + v_y) \approx I_2(x, y) + v_x I_x + v_y I_y,$$

where I_x and I_y are the spatial derivatives of the image (the image gradients). Substituting back into the SSD yields an approximated objective

$$\widetilde{\text{SSD}}(v_x, v_y) = \sum_{x,y} (I_2(x, y) - I_1(x, y) + v_x I_x + v_y I_y)^2.$$

When the two frames are taken with a small time separation, the quantity $I_2(x, y) - I_1(x, y)$ is the change in brightness over time at a fixed pixel—the **temporal derivative** I_t . The core Lucas–Kanade objective is therefore

$$\widetilde{\text{SSD}}(v_x, v_y) = \sum_{x,y} (I_t + v_x I_x + v_y I_y)^2.$$

The Power of Linearization

The Taylor expansion pulls the unknowns (v_x, v_y) out of the image function and turns them into linear coefficients. The approximated objective $\widetilde{\text{SSD}}$ is a quadratic in (v_x, v_y) , so its minimum can be found analytically by setting the gradient to zero—turning a search problem into a closed-form linear system.

5.2 The Normal Equations

Taking partial derivatives of $\widetilde{\text{SSD}}$ with respect to v_x and v_y and setting both to zero yields a system of two linear equations:

$$\begin{aligned} v_x \sum I_x^2 + v_y \sum I_x I_y &= - \sum I_x I_t, \\ v_x \sum I_x I_y + v_y \sum I_y^2 &= - \sum I_y I_t. \end{aligned}$$

In matrix form,

$$A \vec{v} = \vec{b}, \quad A = \begin{pmatrix} \sum I_x^2 & \sum I_x I_y \\ \sum I_x I_y & \sum I_y^2 \end{pmatrix}, \quad \vec{b} = - \begin{pmatrix} \sum I_x I_t \\ \sum I_y I_t \end{pmatrix}.$$

The 2×2 matrix A is built entirely from the spatial gradients of the image patch; the vector $\vec{b}$ carries the temporal information.

Closed-Form Solution

Rather than searching velocity space for the minimum of SSD, the algorithm evaluates image gradients over the patch, assembles A and $\vec{b}$, and solves $\vec{v} = A^{-1} \vec{b}$. This yields the optimal velocity in a single step.

5.3 The Aperture Problem in Matrix Form

The matrix A reveals the aperture problem directly. If the patch contains only a single infinite edge, all image gradients (I_x, I_y) point in the same direction (the normal to the edge). The two rows of A are then linearly dependent, so

$$\text{rank}(A) = 1,$$

and A is not invertible. The equation $A \vec{v} = \vec{b}$ has infinitely many solutions—and these solutions form precisely the constraint line. The algorithm has hit the aperture problem.

If instead the patch contains edges at multiple orientations (a plaid, a corner, or textured content), the gradients point in different directions, the rows of A are linearly independent, and

$$\text{rank}(A) = 2,$$

so A is invertible and $\vec{v} = A^{-1}\vec{b}$ returns the unique IOC velocity.

Same Problem, Three Faces

The aperture problem appears in three equivalent forms in this topic:

- **Psychophysics:** an infinite line's motion is underdetermined—infininitely many velocities produce the same movie.
- **SSD cost landscape:** SSD is flat (zero) along the entire constraint line.
- **Linear algebra:** the matrix A has rank 1, so the normal equations have infinitely many solutions.

A patch with structure in two or more orientations resolves the ambiguity in all three formulations simultaneously: a unique perceived velocity, a unique SSD minimum, a rank-2 matrix.

5.4 Limitations and Practical Extensions

The basic Lucas–Kanade algorithm derived above has a sharp limitation: the Taylor expansion is only accurate when (v_x, v_y) is *very small*. If an object moves by more than a pixel or two between frames, the linearization breaks down and the solution of $A\vec{v} = \vec{b}$ does not correspond to the true motion.

Real-world implementations overcome this with two standard extensions:

1. **Iterative refinement.** Solve for a small velocity, warp I_2 by that estimate, recompute the residual I_t , and solve again. Repeated refinement converges on the true velocity.
2. **Image pyramids (multi-scale).** Downsample both frames repeatedly to lower resolutions. At a coarse scale, a large pixel displacement becomes a small (sub-pixel) displacement, where the Taylor approximation holds. The estimate computed at the coarse scale is then propagated back up to finer scales.

5.5 Lucas–Kanade in Marr's Framework

Stated at Marr's computational level, Lucas–Kanade addresses:

- **Problem:** estimating motion from a sequence of images.
- **Difficulty:** velocity determines a non-linear correspondence between frames.
- **Constraint:** the constant brightness assumption makes the problem solvable, and the small-velocity assumption makes it *linearly* solvable.

Many different algorithms populate the representation-and-algorithm level, but any approach that minimizes SSD under brightness constancy is solving the same computational problem.

6 Summary

This topic bridges human psychophysics and computer-vision algorithms around a single geometric object—the constraint line:

- An infinite line moving with velocity $\vec{v}$ defines a constraint line in velocity space; its existence is the *aperture problem*.
- Two differently-oriented edges give two constraint lines, whose intersection—the *intersection of constraints (IOC)*—is the unique velocity consistent with both measurements.
- Computers estimate motion from the brightness-constancy assumption via SSD; Lucas–Kanade linearizes SSD to turn this into a 2×2 linear system $A\vec{v} = \vec{b}$.
- The aperture problem, zero-SSD flatness, and rank-deficiency of A are three faces of the same phenomenon. In all three cases, adding a second edge orientation resolves the ambiguity.
- Humans and computers face the same uncertainty when estimating motion, and both rely on the same geometric constraints to resolve it.

Quiz

Q1. For any two images generated by two infinite lines of different orientations moving together with velocity $\vec{v}$, the IOC velocity is the minimizer of SSD. True or false?

True. The IOC is the unique intersection of the two constraint lines, and the SSD is zero there (the global minimum).

Q2. For the same setup, the *basic* Lucas–Kanade algorithm will correctly recover the IOC velocity. True or false?

False. Basic Lucas–Kanade relies on a first-order Taylor expansion that is accurate only for very small $\vec{v}$. For large true velocity the linearization fails, and basic Lucas–Kanade does not find the IOC velocity. The iterative, multi-scale (pyramid) version is required.